1장 선형방정식과 LU분해 (Linear Equations and LU decomposition) [^1]

written by Han Suebin

1.1 벡터와 행렬 (Vector and Matrices)

1.1.1 벡터와 행렬 (Vector and Matrices)

우리는 왜 벡터와 행렬을 배울까?

컴퓨터는 '사과'라는 단어를 이해하지 못합니다. 대신 사과의 [빨간 정도, 당도, 무게]를 숫자로 나열한 [90, 80, 200]이라는 묶음을 보고 "이건 사과구나!"라고 이하게됩니다. 이렇게 여러 데이터를 하나의 덩어리로 처리하기 위해 만든 도구가 바로 **벡터와 행렬**입니다.

벡터 (Vector)

- 정의: 벡터는 크기(길이)와 방향을 갖는 기하학적 대상입니다.

벡터는 단순히 숫자 묶음이 아니라, 공간 속에서 **'어디로(방향)', '얼마나(크기)'** 이동할지를 나타내는 화살표와 같습니다. 예를 들어, 2차원 평면에서 오른쪽으로 3칸, 위로 2칸 이동하는 것을 벡터 $\mathbf{v} = [3, 2]$로 표현합니다.

벡터 연산

이렇게 방향과 크기를 가진 벡터가 정의되었다면, 이제 이 화살표들을 더하거나 길이를 조절하는 작업이 필요해지는데요. 이를 위해 다음과 같은 벡터 연산을 진행하게 됩니다.

벡터의 덧셈 (Vector Addition)

- 정의: 두 개의 벡터 $\boldsymbol{x}$와 $\boldsymbol{y}$를 더할 때, 각 성분끼리 더해집니다.

$$\boldsymbol{x} + \boldsymbol{y} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_3 + y_3 \end{bmatrix} \tag{1}$$

어떤 물체가 처음에 $[1, 3, 2]$만큼 이동하고, 나중에 $[4, 1, 5]$만큼 더 이동했다면 최종 위치는 다음과 같습니다.

$$\begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix} = \begin{bmatrix} 1+4 \\ 3+1 \\ 2+5 \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \\ 7 \end{bmatrix} \tag{2}$$

벡터와 스칼라의 곱 (Scalar Multiplication)

- 정의: 벡터 $\mathbf{x}$에 실수(스칼라) c를 곱하면 벡터의 각 성분에 스칼라 c를 곱해 새로운 벡터를 얻습니다.

$$c\boldsymbol{x} = c\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} cx_1 \\ cx_2 \\ cx_3 \end{bmatrix} \tag{3}$$

가고자 하는 방향은 바꾸지 않고, 한 번에 가는 거리(화살표의 길이)만 늘리거나 줄이는 작업입니다. $[2, 1, 4]$라는 방향으로 지금보다 3배 더 멀리 가고 싶다면 다음과 같이 계산합니다.

$$3 \times \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \times 2 \\ 3 \times 1 \\ 3 \times 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \\ 12 \end{bmatrix} \tag{4}$$

행렬(Matrix)

- 정의: 행렬(matrix)은 **행(rows)**과 **열(columns)**로 구성된 원소(수)들의 집합으로, 이를 $m \times n$ 차원 행렬이라고 부릅니다.

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{5}$$

벡터가 하나의 '데이터 묶음'이라면, 행렬은 여러 개의 데이터를 모아놓은 종합 데이터입니다. 예를 들어 다음과 같이 2개의 행과 3개의 열로 이루어진 행렬을 2×3 행렬이라고 부릅니다.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \tag{6}$$

특수 행렬

더 나아가, 모양이 독특해서 계산을 아주 편하게 만들어주는 특수 행렬들이 있습니다. 이들은 데이터 처리에서 핵심적인 역할을 합니다.

- 대각 행렬(Diagonal Matrix)
  주대각선 성분을 제외한 모든 숫자가 0인 행렬입니다.

$$D = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 8 \end{bmatrix} \tag{7}$$

- 항등 행렬 (Identity Matrix)

  대각 행렬 중에서 대각선 성분이 모두 1인 행렬입니다. 기호로는 I라고 씁니다.

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{8}$$

- 상삼각 행렬 (Upper Triangular Matrix)

  대각선을 기준으로 **아래쪽** 성분들이 모두 0인 행렬입니다. 기호로는 U라고 씁니다.

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 5 & 6 \\ 0 & 0 & 9 \end{bmatrix} \tag{9}$$

- 하삼각 행렬 (Lower Triangular Matrix)

  대각선을 기준으로 **위쪽** 성분들이 모두 0인 행렬입니다. 기호로는 L이라고 씁니다.

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 5 & 0 \\ 7 & 8 & 9 \end{bmatrix} \tag{10}$$

- 3중 대각 행렬 (Tridiagonal Matrix)

  주대각선과 그 양쪽 대각 성분만을 가지는 행렬입니다.

$$T = \begin{bmatrix} 2 & 1 & 0 \\ 4 & 3 & 2 \\ 0 & 5 & 6 \end{bmatrix} \tag{11}$$

행렬의 연산

행렬의 덧셈

- 정의: 크기가 같은 두 행렬에서 같은 위치에 있는 원소끼리 더하는 연산입니다.
-

예를 들어, 모든 학생의 국어와 영어 점수를 일괄적으로 합산할 때 사용됩니다. 각 학생의 국어와 영어 점수를 더해 최종 점수를 산출하면 다음과 같습니다.

$$\begin{bmatrix} 80 & 90 \\ 70 & 85 \end{bmatrix} + \begin{bmatrix} 83 & 92 \\ 78 & 97 \end{bmatrix} = \begin{bmatrix} 80+93 & 90+92 \\ 70+78 & 85+97 \end{bmatrix} = \begin{bmatrix} 173 & 182 \\ 148 & 182 \end{bmatrix} \tag{12}$$

행렬의 덧셈은 우리가 평소 쓰는 숫자 더하기와 성질이 매우 비슷합니다.

- 교환법칙 ($A + B = B + A$)

  더하는 순서를 바꿔도 결과는 같습니다

- 결합법칙 ($(A + B) + C = A + (B + C)$)

  세 개 이상의 행렬을 더할 때, 앞에 두 개를 먼저 더하나 뒤에 두 개를 먼저 더하나 결과는 같습니다.

행렬의 뺄셈

- 정의: 크기가 같은 두 행렬에서 같은 위치에 있는 원소끼리 빼는 연산입니다.
-

행렬의 뺄셈을 통해 차이 또는 변화량을 측정합니다. 예를 들어, '기말고사 점수' 행렬에서 '중간고사 점수' 행렬을 빼면, 각 학생이 얼마나 성적이 올랐거나 떨어졌는지(변화량)를 한눈에 알 수 있습니다.

$$\begin{bmatrix} 95 & 80 \\ 70 & 90 \end{bmatrix} - \begin{bmatrix} 90 & 85 \\ 60 & 90 \end{bmatrix} = \begin{bmatrix} 95-90 & 80-85 \\ 70-60 & 90-90 \end{bmatrix} = \begin{bmatrix} 5 & -5 \\ 10 & 0 \end{bmatrix} \tag{13}$$

전치행렬(Transpose Matrix)

- 정의: 행렬의 행(가로)과 열(세로)을 서로 맞바꾼 행렬입니다. 기호로는 A^T라고 씁니다.
-

$$A = \begin{bmatrix} 1 & 100 \\ 2 & 200 \\ 3 & 300 \end{bmatrix} \implies A^T = \begin{bmatrix} 1 & 2 & 3 \\ 100 & 200 & 300 \end{bmatrix} \tag{14}$$

1.1.2 행렬의 곱(Matrix Multiplication)

우리는 왜 행렬의 곱을 배울까?

컴퓨터가 사과를 [빨간 정도, 당도, 무게]라는 벡터로 인식한다는 것을 배웠습니다. 그런데 우리가 이 사과들을 가지고 **'시장 가치'**를 계산하거나, **'품질 등급'**을 매겨야 한다면 어떻게 해야 할까요? 이때 필요한 것이 바로 **행렬의 곱셈**입니다.

행렬의 곱 (Matrix Multiplication)

- 정의: 만약 $A \in \mathbb{R}^{m \times p}$ 이고 $B \in \mathbb{R}^{p \times n}$ 이면, 행렬 곱셈 $C = AB$는 다음과 같이 정의됩니다.

$$c_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj} \tag{15}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} (1 \times 5) + (2 \times 7) & (1 \times 6) + (2 \times 8) \\ (3 \times 5) + (4 \times 7) & (3 \times 6) + (4 \times 8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \tag{16}$$

행렬의 곱셈은 다음과 같은 성질을 가집니다.

- 교환법칙 성립 안 함**($AB \neq BA$)**(가장 중요!)
  두 행렬의 곱셈 순서를 바꾸면 결과가 달라집니다.

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} \text{라고할때,} \tag{17}$$

$$AB\text{계산}: \begin{bmatrix}1 & 2\\0 & 1\end{bmatrix}\begin{bmatrix}2 & 0\\1 & 1\end{bmatrix} = \begin{bmatrix}(2+2) & (0+2)\\(0+1) & (0+1)\end{bmatrix} = \begin{bmatrix}4 & 2\\1 & 1\end{bmatrix} \tag{18}$$

$$BA\text{계산}: \begin{bmatrix}2 & 0\\1 & 1\end{bmatrix}\begin{bmatrix}1 & 2\\0 & 1\end{bmatrix} = \begin{bmatrix}(2+0) & (4+0)\\(1+0) & (2+1)\end{bmatrix} = \begin{bmatrix}2 & 4\\1 & 3\end{bmatrix} \tag{19}$$

- 결합법칙 성립($(AB)C = A(BC)$)

  세 행렬을 곱할 때, 앞에 두 개를 먼저 곱하나 뒤에 두 개를 먼저 곱하나 결과는 똑같습니다.

- 분배법칙 성립($A(B+C) = AB + AC$)

  괄호 밖의 행렬 곱셈을 괄호 안의 더해진 행렬들에 각각 나누어 곱해줄 수 있습니다.

  교환법칙이 성립하지 않는다는 성질 때문에, 분배법칙을 쓸 때도 곱하는 방향을 꼭 지켜야합니다!
  $A(B+C)$는 $AB + AC$라고 써야지, $BA + CA$라고 쓰면 틀립니다.

원소별 곱 (Element-wise Multiplication)

- 정의: 크기가 똑같은 두 행렬에서 같은 위치에 있는 숫자끼리만 단순하게 곱하는 연산입니다. '아다마르 곱(Hadamard Product)'이라고도 합니다.

  행렬 전체를 변환하는 게 아니라, 각 데이터 포인트마다 서로 다른 가중치를 주고 싶을 때 사용합니다.

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} \odot \begin{bmatrix}0 & 5\\2 & 1\end{bmatrix} = \begin{bmatrix}1\times 0 & 2\times 5\\3\times 2 & 4\times 1\end{bmatrix} = \begin{bmatrix}0 & 10\\6 & 4\end{bmatrix} \tag{20}$$

벡터곱(Vector Product)

벡터끼리 곱할 때는 결과가 '스칼라'가 되느냐, '행렬'이 되느냐에 따라 두가지로 나뉩니다.

내적 (Inner Product, Dot Product)

- 정의: 두 n차원 열 벡터 $\mathbf{x}, \mathbf{y}$에 대하여, $\mathbf{x}$를 전치시킨 행 벡터와 열 벡터 $\mathbf{y}$를 곱하여 성분별 곱의 합을 구하는 연산입니다. 결과값은 **스칼라(실수)**입니다.

$$\mathbf{x}^T\mathbf{y} = \begin{bmatrix}x_1 & x_2 & \dots & x_n\end{bmatrix}\begin{bmatrix}y_1\\y_2\\\vdots\\y_n\end{bmatrix} = \sum_{i=1}^{n} x_i y_i = x_1y_1 + x_2y_2 + \cdots + x_ny_n \tag{21}$$

기하학적으로는 한 벡터를 다른 벡터 위로 투영(Projection)시킨 길이를 의미하며, 두 화살표가 비슷한 방향을 향할수록 이 값은 커집니다. 추천 알고리즘에서 유저의 취향과 아이템의 특성이 얼마나 일치하는지 계산할 때 쓰이는 핵심 도구입니다.

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \text{일때,} \tag{22}$$

$$\mathbf{x}^T\mathbf{y} = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = (1 \times 3) + (2 \times 4) = 11 \tag{23}$$

외적 (Outer Product)

- 정의: n차원 열 벡터 $\mathbf{x}$와 m차원 열 벡터 $\mathbf{y}$의 전치 행렬을 곱하여 $n \times m$ 크기의 **행렬**을 만드는 연산입니다.

$$\mathbf{x}\mathbf{y}^T = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \begin{bmatrix} y_1 & y_2 & \cdots & y_m \end{bmatrix} = \begin{bmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_m \\ x_2y_1 & x_2y_2 & \cdots & x_2y_m \\ \vdots & \vdots & \ddots & \vdots \\ x_ny_1 & x_ny_2 & \cdots & x_ny_m \end{bmatrix} \tag{24}$$

내적이 두 정보를 압축하여 '결과값 하나'를 내놓았다면, 외적은 두 정보의 모든 성분을 서로 곱해 '관계표'를 만듭니다.

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \text{일때,} \tag{25}$$

$$\mathbf{x}\mathbf{y}^T = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \end{bmatrix} = \begin{bmatrix} 1 \times 3 & 1 \times 4 \\ 2 \times 3 & 2 \times 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix} \tag{26}$$

1.2 선형 방정식과 LU 분해

1.2.1 선형 방정식의 기하학적 관점

선형대수학의 궁극적인 목표는 $Ax = b$를 푸는 것입니다. 이 수식을 바라보는 두 가지 서로 다른 시각을 이해하는 것이 중요합니다.

행 관점에서의 해석 (Row Picture)

- 정의: 연립 방정식의 각 **개별 방정식**을 공간상의 기하학적 도형(직선이나 평면)으로 보고 해를 찾는 방식입니다.

  3차원 공간($\mathbb{R}^3$)에서 하나의 방정식은 하나의 넓은 **평면**을 의미합니다. 방정식의 해를 구한다는 것은, 여러 개의 평면들이 딱 한 지점에서 만나는 **공통의 교차점**을 찾는 과정입니다.

주어진 3개의 방정식을 각각 평면 P_1, P_2, P_3라고 한다면:

$$\begin{cases} P_1 : u + v + w = 3 \\ P_2 : 2u + 4v - 2w = 4 \\ P_3 : 3u + 2v + w = 6 \end{cases} \tag{27}$$

행 관점에서의 해석 (Row Picture)

- 정의: 행렬 A의 **열 벡터(Column Vector)들의 선형 조합**을 통해 목표 벡터 b를 만들어내는 과정으로 해석하는 방식입니다.

  행렬 A의 각 열은 우리가 가진 데이터이고, 목표 벡터 b는 우리가 완성해야 하는 목표입니다. 이때 해를 구한다는 것은 "가진 데이터를 얼마나 섞어야(u, v, w만큼) 목표에 도달할 수 있을까?"를 알아내는 것과 같습니다.

$$u \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + v \begin{bmatrix} 1 \\ 4 \\ 2 \end{bmatrix} + w \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 6 \end{bmatrix} \tag{28}$$

첫 번째 열 벡터를 u배, 두 번째를 v배, 세 번째를 w배 했을 때 b라는 목표벡터를 만들 수 있는가?"를 묻는 문제입니다.

1.2.2 가우시안 소거법(Gaussian Elimination)

가우스 소거법을 이용한 연립방정식 해 구하기

- 정의: 선형 연립방정식 $Ax = b$를 풀기 위해, 행렬 A에 기본 행 연산(Elementary Row Operations)을 적용하여 상삼각 행렬(Upper Triangular Matrix, U) 형태인 행 사다리꼴(Row Echelon Form)로 변환하는 알고리즘입니다.

  - 기본 행 연산 3가지
    - 한 행에 0이 아닌 상수를 곱한다.
    - 두 행의 위치를 서로 바꾼다.
    - 한 행의 상수를 곱해 다른 행에 더하거나 뺀다.

  첫 번째 식을 이용해 아래 식들의 첫 번째 변수(u)를 없애고, 두 번째 식을 이용해 그 아래 식들의 두 번째 변수(v)를 없애는 식으로 나아갑니다. 이렇게 하면 마지막 식에는 변수 하나만 남게 되어, 마치 계단을 타고 올라오듯 역순으로 정답을 하나씩 맞출 수 있게 됩니다.

1단계 : 전방 소거 (Forward Elimination)

단계	연산 내용	행렬의 상태	해설
0. 초기 상태	**문제 설정**	$\begin{bmatrix} 1 & 1 & 1 & \vdots & 3 \\ 2 & 4 & -2 & \vdots & 4 \\ 3 & 2 & 1 & \vdots & 6 \end{bmatrix}$	Augmented Matrix 형태입니다.
1. 열 소거	$R_2 - 2R_1$ $R_3 - 3R_1$	$\begin{bmatrix} \mathbf{1} & 1 & 1 & \vdots & 3 \\ 0 & 2 & -4 & \vdots & -2 \\ 0 & -1 & -2 & \vdots & -3 \end{bmatrix}$	1열의 피벗 아래를 0으로 만듭니다.
2. 2열 소거	$R_3 + 0.5R_2$	$\begin{bmatrix} 1 & 1 & 1 & \vdots & 3 \\ 0 & 2 & -4 & \vdots & -2 \\ 0 & 0 & \mathbf{-4} & \vdots & -4 \end{bmatrix}$	2열의 피벗 아래를 0으로 만듭니다.

2단계 : 후진 대입 (Back-substitution)

마지막 식부터 거꾸로 미지수를 대입하여 해를 구합니다.

1. 3행: $w = 1$
2. 2행: $2v - 4(1) = -2 \implies v = 1$
3. 1행: $u + 1 + 1 = 3 \implies u = 1$

최종 해: $(u, v, w) = (1, 1, 1)$

LU 분해와 기본 행렬

LU 분해의 기본 원리 및 행렬화

- 정의: 행렬 A를 하삼각행렬(L)과 상삼각행렬(U)의 곱으로 나누는 기법입니다 ($A = LU$).

$$\begin{cases} x + 2y + z = 3 \\ 3x + 4y - z = 1 \\ 2x + 10y + 4z = 10 \end{cases} \rightarrow [A \mid b] = \begin{bmatrix} 1 & 2 & 1 & | & 3 \\ 3 & 4 & -1 & | & 1 \\ 2 & 10 & 4 & | & 10 \end{bmatrix} \tag{29}$$

LU 분해 전방 소거의 행렬화

단계	수행할 행 연산	기본 행렬	결과 첨가 행렬
(1) 1열 소거	$R_2 \leftarrow R_2 - \mathbf{3}R_1$	$E_{21}(3) = \begin{bmatrix} 1 & 0 & 0 \\ \mathbf{3} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 & \mid & 3 \\ 0 & -2 & -4 & \mid & -8 \\ 2 & 10 & 4 & \mid & 10 \end{bmatrix}$

단계	수행할 행 연산	기본 행렬 E_{ij}	결과 첨가 행렬
(2) 1열 소거	$R_3 \leftarrow R_3 - \mathbf{2}R_1$	$E_{31}(2) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \mathbf{2} & 0 & 1 \end{bmatrix}$	$\left[\begin{array}{ccc\|c} 1 & 2 & 1 & 3 \\ 0 & -2 & -4 & -8 \\ 0 & 6 & 2 & 4 \end{array}\right]$
(3) 2열 소거	$R_3 \leftarrow R_3 - (-3)R_2$	$E_{32}(-3) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & \mathbf{-3} & 1 \end{bmatrix}$	$\left[\begin{array}{ccc\|c} 1 & 2 & 1 & 3 \\ 0 & -2 & -4 & -8 \\ 0 & 0 & -10 & -20 \end{array}\right]$

최종 분해 결과

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 2 & -3 & 1 \end{bmatrix}, U = \begin{bmatrix} 1 & 2 & 1 \\ 0 & -2 & -4 \\ 0 & 0 & -10 \end{bmatrix} \tag{30}$$

피벗팅 LU 분해 (LU Decomposition with Pivoting)

- 행 교환의 필요성(Motivation)

가우시안 소거법 수행 중 주대각선 성분(Pivot)이 0이거나, 0에 가까운 매우 작은 값일 경우 연산이 불가능하거나 수치적 불안정성이 발생합니다. 예를 들어, 첫 번째 칸이 0이면 아래 칸들을 소거하기 위한 승수(Multiplier)를 계산할 수 없습니다(0으로 나눌 수 없음). 이때는 행의 순서를 바꾸어 0이 아닌 숫자를 피벗 자리로 가져와야 합니다.

$$A = \begin{bmatrix} \mathbf{0} & 0 & 3 \\ 1 & 1 & 1 \\ 2 & 4 & 5 \end{bmatrix} \tag{31}$$

첫 번째 피벗 a_{11}이 **0**이므로 소거 연산을 시작할 수 없습니다.

- 치환 행렬(P)을 통한 구조적 해결

행 교환(Row Interchange) 연산을 수학적으로 표현하기 위해 단위 행렬(I)의 행 순서를 바꾼 **치환 행렬 (Permutation Matrix, P)**을 도입합니다. 이를 행렬 A의 왼쪽에 곱하면 행의 위치가 바뀐 새로운 행렬 PA를 얻게 됩니다.

1. 치환 (Pivoting): A의 1행과 2행을 교환하기 위해 치환 행렬 P를 곱합니다.

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{32}$$

$$PA = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 3 \\ 1 & 1 & 1 \\ 2 & 4 & 5 \end{bmatrix} = \begin{bmatrix} \mathbf{1} & 1 & 1 \\ 0 & 0 & 3 \\ 2 & 4 & 5 \end{bmatrix} \tag{33}$$

2. 분해 (LU Decomposition): 이제 첫 번째 피벗이 **1**이 되었으므로, 행이 교환된 PA를 대상으로 일반적인 LU 분해를 수행할 수 있습니다. 최종 식은 $PA = LU$ 형태가 됩니다.

- 피벗팅 LU 분해의 전체 알고리즘

단계	연산 식	전문적 해설
1. 시스템 분해	$PA = LU$	치환 행렬(P)을 사용하여 행을 적절히 교환한 뒤, 안정적인 하삼각행렬(L)과 상삼각행렬(U)로 분해합니다.
2. 결과값 재정렬	$Pb = b'$	행렬의 순서가 바뀌었으므로, 목표 벡터 b에도 동일한 치환 행렬(P)을 곱하여 순서를 일치시킵니다.
3. 전방 대입	$Ly = Pb$	결정 구조인 L을 이용하여, 위에서 아래로 정답을 채워나가며 임시 벡터 y를 계산합니다.
4. 후방 대입	$Ux = y$	정제된 모델인 U에 임시 벡터 y를 대입하여, 아래에서 위로 최종 해 x를 도출합니다.

다음과 같은 시스템 $Ax = b$를 해결야 할 때,

$$\begin{bmatrix} 0 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 4 \\ 10 \end{bmatrix} \tag{34}$$

단계	연산 식	실제 계산 과정	결과 및 해설
1. 치환 (Pivoting)	PA	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} \mathbf{3} & 4 \\ 0 & 2 \end{bmatrix}$ 1행 피벗이 0이므로 행을 교체합니다.
2. LU 분해	$PA = LU$	$L = I, U = PA$	소거할 성분이 없어 즉시 분해됩니다.
3. 결과값 재정렬	$Pb = b'$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ 10 \end{bmatrix}$	$\begin{bmatrix} 10 \\ 4 \end{bmatrix}$ 목표 벡터 b의 순서를 동기화합니다.
4. 전방 대입	$Ly = Pb$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 10 \\ 4 \end{bmatrix}$	$y_1 = 10, y_2 = 4$ 임시 해를 구합니다.
5. 후방 대입	$Ux = y$	$\begin{bmatrix} 3 & 4 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 10 \\ 4 \end{bmatrix}$	$v = 2, u = 2/3$ 최종 해를 도출합니다.

1.3 행렬식(Determinants)

1.3.1 소개 (Introduction)

- 정의: 정방행렬($n \times n$ 행렬)을 하나의 실수로 대응시키는 함수로, $\det(A)$ 또는 $|A|$로 표기합니다.

  이 숫자 하나를 보면 이 행렬이 역행렬을 가질 수 있는지, 아니면 데이터를 찌그러뜨려 정보를 손실시키는를 알 수 있습니다.

1.3.2 행렬식의 특성 (The Properties of the Determinant)

1. 삼각행렬의 행렬식 (Determinant of Triangular Matrices)

- 정의: 상삼각 또는 하삼각행렬의 행렬식은 주대각선 원소들의 곱과 같습니다.

$$\det(\mathbf{A}) = a_{11}a_{22}\cdots a_{nn} \tag{35}$$

$$A = \begin{bmatrix} \mathbf{2} & 3 \\ 0 & \mathbf{5} \end{bmatrix} \tag{36}$$

$$\text{행렬식} : 2 \times 5 = 10\text{입니다.}$$

2. 역행렬과 행렬식

- 정의: 행렬 A가 역행렬을 가지기 위한 필요충분조건은 $\det(A) \neq 0$입니다.

  행렬식이 0이면 그 행렬은 '특이 행렬(Singular Matrix)'이 되어 정보를 되돌릴 수(역행렬) 없습니다. 연립방정식으로 치면 답이 없거나 무수히 많은 불안정한 상태가 됩니다.

3. 행렬식의 기하학적 의미

- 정의: 행렬이 가하는 변환의 확장/축소 비율(Scale Factor)을 나타냅니다.
  - 2차원: 행렬의 열벡터들이 만드는 평행사변형의 면적입니다.
  - 3차원:열벡터들이 만드는 평행육면체의 부피입니다.
  - 핵심: 만약 행렬식이 0이라면? 면적이나 부피가 0이라는 뜻입니다. 즉, 평면이 선으로 찌그러지거나 공간이 평면으로 눌려버려 정보가 소실되었다는 의미입니다.

4. 행렬식의 곱의 법칙

- 정의: 두 행렬을 곱한 결과의 행렬식은 각각의 행렬식을 곱한 것과 같습니다.

$$\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B}) \tag{37}$$

  행렬 A가 공간을 2배 키우고, B가 3배 키운다면, 둘을 연달아 적용(AB)하면 공간은 총 6배 커진다는 상식적인 법칙입니다. (행렬 곱은 순서가 중요하지만, 행렬식은 곱하는 순서를 바꿔도 결과가 같습니다!)

5. 대각합 (Trace)의 성질과 곱의 법칙

- 정의: 정방행렬의 주대각선 원소들을 모두 더한 값입니다.

$$\mathrm{tr}(\mathbf{A}) = a_{11} + a_{22} + \cdots + a_{nn} \tag{38}$$

대각합 역시 행렬 곱셈의 순서를 바꿔도 그 합이 변하지 않는 교환법칙이 성립합니다.

$$\mathrm{tr}(\mathbf{AB}) = \mathrm{tr}(\mathbf{BA}) \tag{39}$$

두 행렬 $A = \begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix}$, $B = \begin{bmatrix} 4 & 5 \\ 6 & 1 \end{bmatrix}$에 대하여:

구분	결과 행렬	대각합(tr) 계산
AB 결과	$\begin{bmatrix} \mathbf{16} & 7 \\ 12 & \mathbf{15} \end{bmatrix}$	$16 + 15 = \mathbf{31}$
BA 결과	$\begin{bmatrix} \mathbf{19} & 8 \\ 9 & \mathbf{12} \end{bmatrix}$	$19 + 12 = \mathbf{31}$

1.4 콜레스키 분해(The Cholesky factorization)

왜 콜레스키 분해인가?

LU 분해는 모든 정방행렬에 적용할 수 있는 범용 도구였습니다. 하지만 우리가 다루는 데이터가 **'대칭(Symmetric)'**이고 **'안정적(Positive Definite)'**이라면, 굳이 무거운 LU 분해를 쓸 필요가 없습니다. 콜레스키 분해는 이 두 특성을 이용해 연산 속도를 2배 높이고 메모리 사용량을 절반으로 줄여 줍니다.

1.4.1 양의 정부호 행렬(Symmetric positive definite matrix)

- **대칭 행렬 (Symmetric Matrix)** : 행렬 A가 주대각선을 기준으로 대칭인 상태입니다

$$\mathbf{A} = \mathbf{A}^T \tag{40}$$

- **양의 정부호(Positive Definite, PD) 행렬** : 임의의 영벡터가 아닌 벡터 x에 대해, 이차 형식 $x^T Ax$가 항상 **양수(> 0)**가 되어야 합니다.

$$\text{For all } \mathbf{x} \neq \mathbf{0}, \quad \mathbf{x}^{\mathrm{T}}\mathbf{Ax} > 0 \tag{41}$$

- **양의 준정부호(Positive Semi-Definite, PSD) 행렬** : 대칭 행렬 $\mathbf{A}$에 대해, 이차 형식 $\mathbf{x}^{\mathrm{T}}\mathbf{Ax}$의 값이 0보다 크거나 같을 때($\geq 0$), $\mathbf{A}$를 양의 준정부호 행렬이라고 합니다.

$$\text{For all } \mathbf{x} \neq \mathbf{0}, \quad \mathbf{x}^{\mathrm{T}}\mathbf{Ax} \geq 0 \tag{42}$$

1.4.2 콜레스키 분해 (The Cholesky factorization)

- 정의: 대칭 양의 정부호 행렬 A를 하삼각행렬 L과 그 전치행렬 L^T의 곱으로 분해하는 것입니다.

$$\mathbf{A} = \mathbf{L}\mathbf{L}^{\mathrm{T}} \tag{43}$$

일반적인 LU 분해는 L과 U가 서로 다르게 생겼지만, 콜레스키 분해는 A의 대칭성을 활용해 U 자리에 L의 거울 이미지인 L^T를 그대로 사용합니다. 덕분에 저장 공간과 계산량을 절반으로 줄일 수 있습니다.

하지만 콜레스키 분해를 쓰기 위해서는 분해하고자 하는 행렬이 다음 두 가지 자격 요건을 갖추어야 합니다.
$\mathbf{A} = \mathbf{L}\mathbf{L}^{\mathrm{T}}$로 분해되기 위해서는 $\mathbf{A}$가 **1)대칭 행렬**이면서 **2)양의 준정부호 행렬(Positive Semi-Definite)**이어야 합니다.

- 왜 양의 준정부호(PSD)여야만 하는가?

  $A = LL^T$라고 가정하고 이차 형식을 계산해 보면 그 이유가 명확해집니다.

$$x^T A x = x^T (LL^T) x = (L^T x)^T (L^T x) = \|L^T x\|_2^2 \tag{44}$$

벡터의 길이(L2-norm)의 제곱은 수학적으로 **항상 0보다 크거나 같습니다(≥ 0)**. 따라서 $A = LL^T$라는 깔끔한 형태로 분해되기 위해서는, 원본 행렬 A 자체가 애초에 $x^T A x \geq 0$을 만족하는 **양의 준정부호(PSD)** 성질을 가지고 있어야만 모순이 생기지 않습니다.

LU 분해 vs 콜레스키 분해 비교

구분	일반 LU 분해 ($A = LU$)	콜레스키 분해 ($A = LL^T$)
적용 대상	모든 정방행렬 (피벗팅 필요할 수 있음)	**대칭 양의 정부호(SPD)** 행렬만 가능
계산 효율	표준 연산량 (약 $\frac{2}{3}n^3$)	**절반의 연산량** (약 $\frac{1}{3}n^3$)
메모리 저장	L과 U 두 개를 모두 저장	L **하나만 저장**
수치적 안정성	피벗팅 없으면 불안정할 수 있음	피벗팅 없이도 **극도로 안정적**

콜레스키 분해의 계산

$$A = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix} \tag{45}$$

STEP 1: SPD 성질 확인

항목	계산 과정	결과
대칭성	$a_{12} = a_{21} = 2$	**대칭**
정부호성	$\det[4] = 4 > 0, \det(A) = 16 > 0$	**양의 정부호**

STEP 2: 원소별 계산 과정

위치	비교 식	계산	결과
(1, 1)	$l_{11}^2 = 4$	$\sqrt{4}$	$l_{11} = 2$
(2, 1)	$l_{21}l_{11} = 2$	$l_{21} \times 2 = 2$	$l_{21} = 1$
(2, 2)	$l_{21}^2 + l_{22}^2 = 5$	$1^2 + l_{22}^2 = 5$	$l_{22} = 2$

최종 분해 결과:

$$L = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \quad L^T = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix} \tag{46}$$

1.5 1장 연습문제 및 풀이

I. 연습문제 (Problems)

문제 1. 벡터 연산 및 행렬 곱 (Maxtrix Multiplication)

두 벡터 $\mathbf{x}$와 $\mathbf{y}$, 행렬 $\mathbf{A}$가 다음과 같을 때, 다음 연산을 수행하시오.

$$\mathbf{x} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

1. 벡터의 덧셈 $\mathbf{x} + \mathbf{y}$
2. 벡터의 내적 $\mathbf{x}^{\mathrm{T}}\mathbf{y}$
3. 벡터의 외적 $\mathbf{x}\mathbf{y}^{\mathrm{T}}$
4. 행렬 곱셈 $\mathbf{A}(\mathbf{x} + \mathbf{y})$

문제 2. 가우시안 소거법 (Gaussian Elimination)

다음 연립 선형 방정식을 **가우시안 소거법**을 이용하여 풀고, 그 해 $\mathbf{x} = (x, y)$를 구하시오.

$$\begin{aligned} 2x + y &= 7 \\ 4x + 5y &= 17 \end{aligned}$$

문제 3. LU분해의 활용 및 효율성

$\mathbf{LU}$ 분해를 사용하는 가장 주된 목적과, 이 방법이 일반적인 가우시안 소거법보다 효율적인 이유를 **시간 복잡도**와 연관 지어 서술하시오.

문제 4. LU 분해

다음 행렬 $\mathbf{A}$에 대해 가우시안 소거법의 전방 소거를 수행하여 **하삼각행렬 $\mathbf{L}$**과 **상삼각행렬 $\mathbf{U}$**를 구하시오.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 6 & 4 \\ 3 & 8 & 9 \end{bmatrix} \tag{47}$$

문제 5. 양의 정부호 행렬(SPD) 판별

다음 행렬 $\mathbf{A}$가 **양의 정부호 행렬(SPD)**인지 판별하는 방법을 **주부분 행렬식(Determinant)**을 이용하여 서술하고, 실제로 $\mathbf{SPD}$인지 판단하시오.

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} \tag{48}$$

II. 해설 (Solutions)

해설 1.

$$\mathbf{x} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$
$$\mathbf{x} + \mathbf{y} = \begin{bmatrix} 4+2 \\ 1+5 \end{bmatrix} = \begin{bmatrix} 6 \\ 6 \end{bmatrix}$$
$$\mathbf{x}^{\mathrm{T}}\mathbf{y} = (4 \cdot 2) + (1 \cdot 5) = 13$$
$$\mathbf{x}\mathbf{y}^{\mathrm{T}} = \begin{bmatrix} 4 \\ 1 \end{bmatrix} \begin{bmatrix} 2 & 5 \end{bmatrix} = \begin{bmatrix} 8 & 20 \\ 2 & 5 \end{bmatrix}$$
$$\mathbf{A}(\mathbf{x} + \mathbf{y}) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 6 \\ 6 \end{bmatrix} = \begin{bmatrix} 18 \\ 42 \end{bmatrix}$$

해설 2.

1. 전방 소거 (Forward Elimination): 상삼각행렬 형태로 변환

$$\left[\begin{array}{cc|c} 2 & 1 & 7 \\ 4 & 5 & 17 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \left[\begin{array}{cc|c} 2 & 1 & 7 \\ 0 & 3 & 3 \end{array}\right]$$

2. 후진 대입 (Back-substitution):
   - $3y = 3 \implies \mathbf{y = 1}$
   - $2x + 1 = 7 \implies \mathbf{x = 3}$

최종 해: $\mathbf{x} = 3, y = 1$ 입니다.

해설 3.

1. 주된 목적: $\mathbf{Ax} = \mathbf{b}$ 형태의 연립 선형 방정식을 $\mathbf{Ly} = \mathbf{b}$와 $\mathbf{Ux} = \mathbf{y}$라는 두 단계의 삼각행렬 문제로 분리하여 해결합니다.
2. 효율적인 이유: $\mathbf{LU}$ 분해는 행렬 $\mathbf{A}$를 분해하는 초기 비용은 $\sim n^3$이지만, 분해 후 해를 구하는 반복 비용은 $\sim 2n^2$에 불과합니다. 따라서 $\mathbf{A}$가 고정된 채 $\mathbf{b}$만 바뀌는 반복적인 계산에서 일반 소거법($\sim n^3$ 매번 계산)보다 압도적으로 빠른 속도를 제공합니다.

해설 4. LU 분해

1. **전방 소거 및 $\mathbf{U}$ 구하기:**

연산	승수(m_{ij})	결과 $\mathbf{A}$
$R_2 \leftarrow R_2 - \mathbf{2}R_1$	$m_{21} = 2$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 2 \\ 3 & 8 & 9 \end{bmatrix}$
$R_3 \leftarrow R_3 - \mathbf{3}R_1$	$m_{31} = 3$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 2 \\ 0 & 2 & 6 \end{bmatrix}$
$R_3 \leftarrow R_3 - \mathbf{1}R_2$	$m_{32} = 1$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 2 \\ 0 & 0 & 4 \end{bmatrix} = \mathbf{U}$

2. **$\mathbf{L}$ 구하기:** $\mathbf{L}$은 주대각 원소가 1이고 하삼각 성분에 사용된 승수(m_{ij})를 사용하여 만듭니다.

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 1 & 1 \end{bmatrix} \tag{49}$$

해설 5. 양의 정부호 행렬 (SPD) 판별

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} \tag{50}$$

판별 방법:대칭성을 확인한 후, 모든 주부분 행렬식($\det$)이 양수(> 0)인지 확인합니다.

1. **대칭성 확인:** $\mathbf{A} = \mathbf{A}^{\mathrm{T}}$이므로 대칭 행렬입니다.
2. **주부분 행렬식 확인:**
   - $\det(\mathbf{A}_1)$: $\det([2]) = 2 > 0$
   - $\det(\mathbf{A}_2)$: $\det \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = 4 - 1 = 3 > 0$
   - $\det(\mathbf{A}_3)$: $\det(\mathbf{A}) = 2(4-1) - 1(2-0) + 0 = 4 > 0$

판단: 모든 주부분 행렬식이 양수(> 0)이므로, 행렬 $\mathbf{A}$는 양의 정부호 행렬(SPD)이 맞습니다.